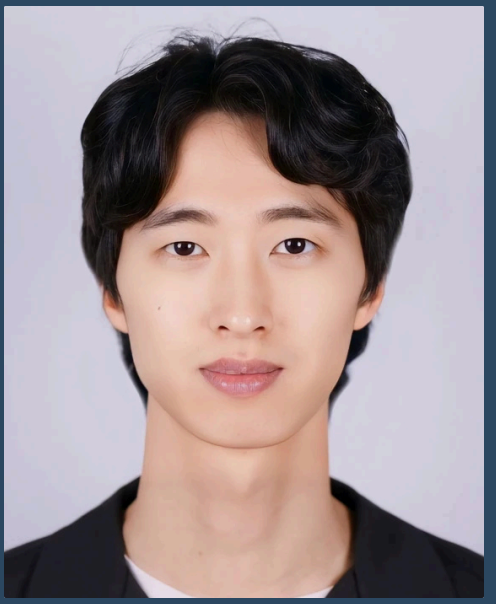
로봇 제어 · 비전 · 임베디드 신입

정승호 SEUNGHO JUNG

비전으로 로봇팔 동작을 보정하고, 그 모델을 Raspberry Pi5에 올렸습니다

3095069@naver.com



프로젝트와 실습에서 사용한 기술

01 제어·임베디드

C · C++ · STM32

FreeRTOS · PLC

02 로봇·비전

Python · ROS2 · OpenCV

YOLO · TFLite

03 설계·개발 도구

SolidWorks · Ansys · MATLAB

MuJoCo · Isaac Sim

학력과 교육, 자격 사항

기계공학 전공 지식을 바탕으로 로봇 제작과 제어를 학습했습니다

학력·교육

2026.04 – 2026.11 수료

대한상공회의소 서울기술교육센터

AI융합 로봇 SW개발자 3기

Python·Linux AI 프로그래밍
ROS/ROS2 통신·제어
Cortex-M·PLC·산업용 네트워크

2026.02 졸업

순천향대학교 기계공학과

학점 3.65 / 4.5

2019.02 졸업

대성고등학교

자격·어학

TOEIC 705점

2025.12

운전면허 1종 보통

2019.08 · 경찰청

초경량비행장치조종자

2019.09 · 한국교통안전공단

지그 위치에 동작을 맞추고 부품 안착까지 검사하는 로봇팔

Raspberry Pi5에서 AI·OpenCV·CAD로 지그 위치와 부품 상태를 확인했습니다

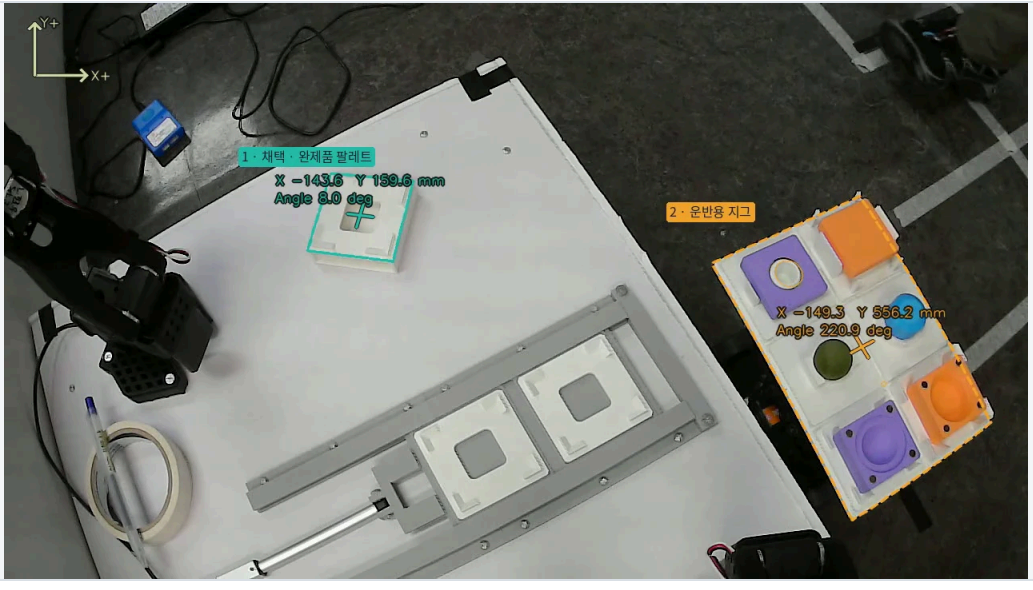


그림 1. 지그 이동 후 위치·방향을 다시 채택하는 화면

운반 로봇의 정차 위치가 달라지고 부품이 서로 가려도
저장한 동작으로 조립·하차 공정을 이어가게 했습니다
카메라로 지그를 다시 측정해 동작 목표를 보정하고,
부품 종류와 안착 상태를 검사했습니다

담당 역할

로봇팔 2대의 비전 보정·티칭 GUI·Raspberry Pi5 연동
학습 데이터·모델 비교와 실행 기준 검증

Python · OpenCV | YOLO · CAD 정합 | FK · IK | ROS2 Action | MuJoCo · Isaac Sim

기간	공정	장비	인식 대상
2026.09 – 2026.11	지그 도착 → 조립 → 완제품 하차	SO-101 2대 · Raspberry Pi5	부품·지그 13종

AI 인식과 설계 형상 맞춤으로 지그·팔레트 위치 확정

외곽선만으로는 가려진 팔레트를 놓쳐, AI가 찾은 영역을 설계 형상에 다시 맞추고 위치·방향이 맞을 때만 채택했습니다



그림 2. 운반용 지그(위)와 완제품 팔레트(아래)의 위치를 찾는 과정

운반용 지그 · 360° 방향 검증

AI 후보 영역을 외곽·격자·원형 특징으로 검증
첫 3관측이 **0.5mm·0.5°** 이내면 바로 채택
아니면 4초 판단 후 과반 그룹 중앙값, 5회 실패 시 중단

완제품 팔레트 · CAD 정합

AI가 찾은 영역에 설계 형상을 맞춰
가려진 부분까지 위치 계산
285장 중 284장에서 팔레트를 검출

팔레트 검출률	외곽선 검출	AI 인식 + 외곽선	AI 인식 + 설계 형상 맞춤 · 채택
촬영 사진 285장 · 최초 검출 시	**79.3%** (226/285장)	**80.7%** (230/285장)	**99.6%** (284/285장)

부품 종류·안착 검사와 Raspberry Pi5 경량화

OpenCV만으로 생긴 구분 오류를 없애려고 종류와 안착 검사를 나누고, Raspberry Pi5에 맞춰 실행 형식과 모델 크기를 정했습니다

부품 검사

AI 라벨 · 무엇이 있는가

A/B 부품·조립 상태를 구분해 검사 위치에 연결

CAD 검사 · 제자리에 있는가

설계 형상·위치·방향으로 안착 상태 확인

같은 검사 위치에서 비교한 결과

방식	부품 구분	평균 처리
OpenCV	247/249	0.937초
AI 라벨 + CAD 채택	**249/249**	**1.592초**

부품 구분 오류 2건을 없애 **249개 모두 정확히 구분**

오안착 검출 사례	처리	기록
이탈 7.8mm > 허용 5mm	안착 불합격 · 작업 중단	판정 영상·오류 기록 저장

Raspberry Pi5 실행 기준

순차 학습 채택

최저 클래스 IoU **96.72%**

평균 IoU가 비슷한 혼합·순차 중 편차 0.60으로 가장 안정적

FP16 채택

처리 시간 **61%** 감소

1.341초 → 0.523초 · 원본 대비 IoU 차이 0.02점

YOLO26n 채택

처리 시간 **53%** 감소

s 대비 IoU 차이 0.15점 · 메모리 38% 절감

실물 31장·169개 · 신뢰도 50% · 처리 시간은 Pi5 중앙값

스마트폰으로 조작하는 이동형 로봇

4륜 이동체와 로봇팔을 결합해 스마트폰으로 주행과 물품 회수 동작을 제어했습니다

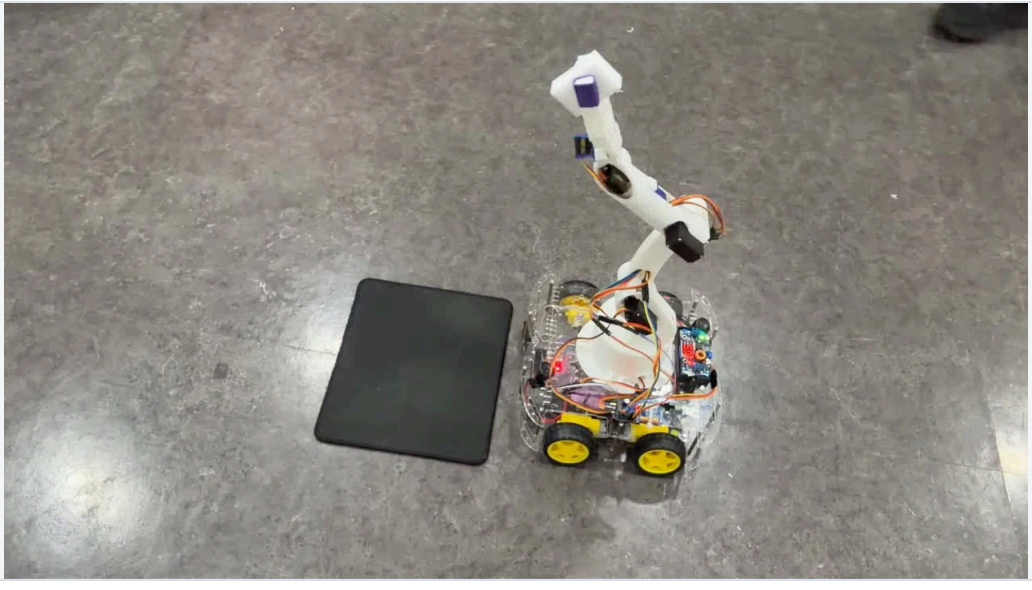


그림 3. 원격 물품 회수·운반 로봇 시연품

이동체와 로봇팔을 결합한 물품 회수·운반 시연품을 만들었습니다
앱으로 주행과 관절을 수동 조작하고, 저장한 동작을 재생하게 했습니다

담당 역할

프로젝트 리더 · 로봇팔 제작·제어, STM32 펌웨어 구축·통합
통신 패킷 규격 설계와 앱 연동

STM32F411 | C · FreeRTOS | Flutter | Bluetooth

수행 시기	제어 보드	구동 구성	구현 범위
2026.07 · 1주	**STM32F411**	**4WD · 서보 6개**	**수동 조작 · 티칭**

16바이트 통신 규격과 구동 문제 개선

16바이트 규격으로 앱과 펌웨어를 연결하고, 부하 시 떨림과 급격한 움직임을 줄였습니다

16바이트 명령·응답

주행·로봇팔·티칭·설정을 모드로 구분

앱 요청과 펌웨어 응답을 연결해 처리 상태 확인

서보 제어와 동작 저장

PCA9685로 서보 6개 제어

Flash에 티칭·설정 저장, 최대 **12개 시퀀스 × 30개 웨이포인트**

동작 간 간섭 방지

로봇팔 이동·Flash 작업 중 주행 잠금

500ms 동안 유효한 주행 명령이 없으면 이동체 정지

문제	개선 내용
부하가 걸릴 때의 떨림	전원부 커패시터 추가, 토크 여유가 있는 서보로 교체 **부하 시 로봇팔 떨림 감소**
관절의 급격한 움직임	**20ms 주기 S-curve**로 관절 구동
관절 원점·가동 범위 보정	**−90°·0°·+90° 3점 보정**으로 원점·리미트 설정 서보별 실측 펄스값을 저장해 관절 제어에 반영

객체 탐지와 지면 분할을 이용한 보행 보조

Raspberry Pi5에서 영상 처리, 객체 탐지, 지면 분할을 함께 실행했습니다



그림 4. Raspberry Pi5에서 실행한 객체·지면 인식 화면

영상에서 사람·차량·장애물을 탐지하고 인도와 차도를 구분했습니다
보행 경로에 가까운 대상을 판단해 음성으로 알려 주게 했습니다

> **담당 역할**
> 학습 데이터 라벨 통합, 모델 학습·변환, 실행 환경 구성
> 보행·차량 ROI와 알림 판단 로직 구현, 녹화 영상으로 동작 확인

Python · OpenCV | YOLO26n | TFLite INT8

수행 시기	실행 장치	객체 라벨 통합	지면 라벨 통합
2026.05 · 11일	Raspberry Pi5	29종 → 10종	23종 → 7종

Raspberry Pi5 실행 환경에 맞춘 조정

Raspberry Pi5 한 대에서 두 모델을 함께 돌리기 위해 입력과 모델을 줄이고, 판단 영역을 보행과 차량으로 나눴습니다

입력 해상도 축소와 모델 변환

입력 해상도 **1920×1080 → 512×288**, 모델은 **INT8 TFLite**로 변환

보행·차량 ROI 분리

보행 ROI는 발 앞쪽인 화면 하단 중앙에 배치
옆에서 접근하는 차량은 더 넓은 **차량 ROI**로 판단

녹화 영상 기반 동작 시험

녹화 영상을 재생하며 추론 동작 확인
추론 빈도는 **객체 4fps, 지면 3fps**로 제한

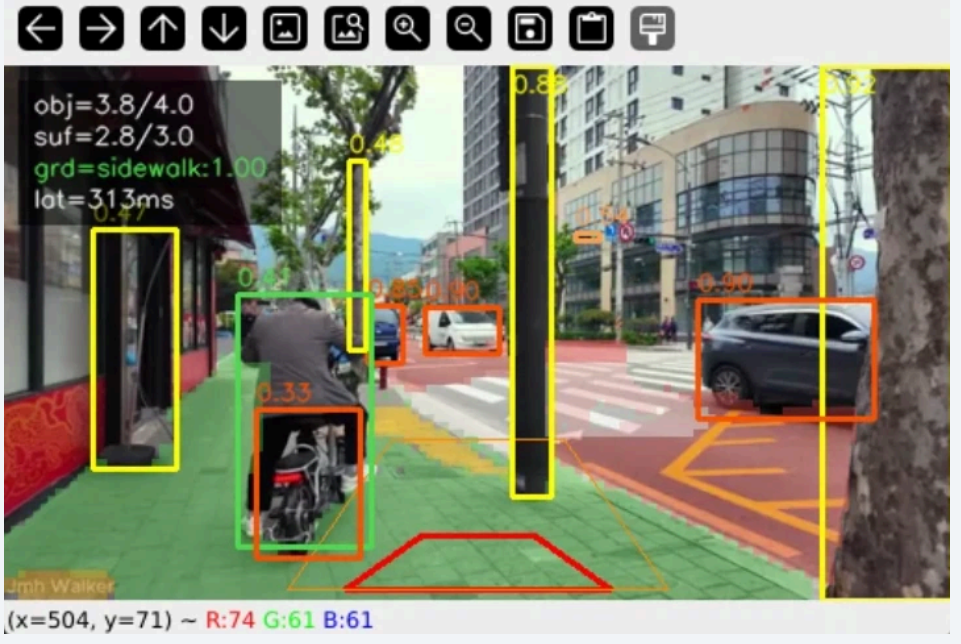


그림 5. 빨간색: 보행 경로 · 주황색: 차량 접근 영역